

HUNDRED DAYS OF CODE

DAY 16

QUESTION: Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether its digits are in increasing order from left to right.
- Check whether its digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

ALGORITHM:

Main Algorithm

Step 1: Start.

Step 2: Display the menu containing four operations and an Exit option.

Step 3: Read the user's choice.

Step 4: Use switch-case to call the appropriate function:

- Choice 1 → checkIncreasing()
- Choice 2 → checkDecreasing()
- Choice 3 → findLargestSmallest()
- Choice 4 → findSecondLargestSmallest()
- Choice 5 → Exit

Step 5: Perform the selected operation and display the result.

Step 6: Repeat the menu until the user chooses Exit.

Step 7: Stop.

Algorithm for Option 1: Increasing Order

Step 1: Read a positive integer.

Step 2: Extract its digits and compare every digit with the digit immediately before it.

Step 3: Since digits are extracted from right to left, compare accordingly.

HUNDRED DAYS OF CODE

DAY 16

Step 4: If every digit is greater than or equal to the digit on its left, the digits are in increasing order.

Step 5: Otherwise, the digits are not in increasing order.

Step 6: Display the result.

Algorithm for Option 2: Decreasing Order

Step 1: Read a positive integer.

Step 2: Extract the digits from right to left.

Step 3: Compare each pair of adjacent digits.

Step 4: If every digit is less than or equal to the digit on its left, the digits are in decreasing order.

Step 5: Otherwise, the digits are not in decreasing order.

Step 6: Display the result.

Algorithm for Option 3: Largest and Smallest Digit

Step 1: Read a positive integer.

Step 2: Initialize largest = 0 and smallest = 9.

Step 3: Extract each digit.

Step 4: Compare the digit with largest and smallest.

Step 5: Update the values whenever necessary.

Step 6: Display the largest and smallest digits.

Algorithm for Option 4: Second Largest and Second Smallest

Step 1: Read a positive integer.

Step 2: Initialize variables for the largest, second largest, smallest, and second smallest digits.

Step 3: Extract each digit one at a time.

Step 4: Update the largest and second-largest distinct digits.

HUNDRED DAYS OF CODE

DAY 16

Step 5: Update the smallest and second-smallest distinct digits.

Step 6: If at least two distinct digits are not available, display an appropriate message.

Step 7: Otherwise, display the second largest and second smallest digits.

PSEUDOCODE:

Main Program

BEGIN

REPEAT

DISPLAY "===== MENU ====="

DISPLAY "1. Check increasing order"

DISPLAY "2. Check decreasing order"

DISPLAY "3. Find largest and smallest digit"

DISPLAY "4. Find second largest and second smallest digit"

DISPLAY "5. Exit"

READ choice

SWITCH choice

CASE 1:

CALL checkIncreasing()

BREAK

CASE 2:

CALL checkDecreasing()

BREAK

HUNDRED DAYS OF CODE

DAY 16

CASE 3:

CALL findLargestSmallest()

BREAK

CASE 4:

CALL findSecondLargestSmallest()

BREAK

CASE 5:

DISPLAY "Program terminated."

BREAK

DEFAULT:

DISPLAY "Invalid choice! Please try again."

END SWITCH

UNTIL choice = 5

END

Function 1: Check Increasing Order

FUNCTION checkIncreasing()

READ positive integer n

HUNDRED DAYS OF CODE

DAY 16

SET previous = $n \bmod 10$

SET $n = n \div 10$

SET increasing = TRUE

WHILE $n > 0$

 SET digit = $n \bmod 10$

 IF digit > previous THEN

 SET increasing = FALSE

 BREAK

 END IF

 SET previous = digit

 SET $n = n \div 10$

END WHILE

IF increasing = TRUE THEN

 DISPLAY "The digits are in increasing order from left to right."

ELSE

 DISPLAY "The digits are not in increasing order from left to right."

END IF

END FUNCTION

Function 2: Check Decreasing Order

HUNDRED DAYS OF CODE

DAY 16

FUNCTION checkDecreasing()

READ positive integer n

SET previous = n MOD 10

SET n = n DIV 10

SET decreasing = TRUE

WHILE n > 0

SET digit = n MOD 10

IF digit < previous THEN

SET decreasing = FALSE

BREAK

END IF

SET previous = digit

SET n = n DIV 10

END WHILE

IF decreasing = TRUE THEN

DISPLAY "The digits are in decreasing order from left to right."

ELSE

DISPLAY "The digits are not in decreasing order from left to right."

END IF

HUNDRED DAYS OF CODE

DAY 16

END FUNCTION

Function 3: Find Largest and Smallest Digit

FUNCTION findLargestSmallest()

 READ positive integer n

 SET largest = 0

 SET smallest = 9

 WHILE n > 0

 SET digit = n MOD 10

 IF digit > largest THEN

 SET largest = digit

 END IF

 IF digit < smallest THEN

 SET smallest = digit

 END IF

 SET n = n DIV 10

 END WHILE

HUNDRED DAYS OF CODE

DAY 16

DISPLAY "Largest digit = ", largest

DISPLAY "Smallest digit = ", smallest

END FUNCTION

Function 4: Find Second Largest and Second Smallest Digit

FUNCTION findSecondLargestSmallest()

READ positive integer n

SET largest = -1

SET secondLargest = -1

SET smallest = 10

SET secondSmallest = 10

WHILE n > 0

SET digit = n MOD 10

IF digit > largest THEN

SET secondLargest = largest

SET largest = digit

ELSE IF digit > secondLargest AND digit != largest THEN

SET secondLargest = digit

END IF

HUNDRED DAYS OF CODE

DAY 16

```
IF digit < smallest THEN
```

```
    SET secondSmallest = smallest
```

```
    SET smallest = digit
```

```
ELSE IF digit < secondSmallest AND digit != smallest THEN
```

```
    SET secondSmallest = digit
```

```
END IF
```

```
SET n = n DIV 10
```

```
END WHILE
```

```
IF secondLargest = -1 OR secondSmallest = 10 THEN
```

```
    DISPLAY "At least two distinct digits are required."
```

```
ELSE
```

```
    DISPLAY "Second largest digit = ", secondLargest
```

```
    DISPLAY "Second smallest digit = ", secondSmallest
```

```
END IF
```

```
END FUNCTION
```

4. C Program

```
#include <stdio.h>
```

```
/* Function to check increasing order */
```

HUNDRED DAYS OF CODE

DAY 16

```
void checkIncreasing()
{
    int n, digit, previous, increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    previous = n % 10;
    n = n / 10;

    while (n > 0)
    {
        digit = n % 10;

        if (digit > previous)
        {
            increasing = 0;
            break;
        }

        previous = digit;
        n = n / 10;
    }

    if (increasing)
        printf("The digits are in increasing order from left to right.\n");
    else
```

HUNDRED DAYS OF CODE

DAY 16

```
printf("The digits are not in increasing order from left to right.\n");  
}
```

```
/* Function to check decreasing order */
```

```
void checkDecreasing()
```

```
{
```

```
    int n, digit, previous, decreasing = 1;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    previous = n % 10;
```

```
    n = n / 10;
```

```
    while (n > 0)
```

```
    {
```

```
        digit = n % 10;
```

```
        if (digit < previous)
```

```
        {
```

```
            decreasing = 0;
```

```
            break;
```

```
        }
```

```
        previous = digit;
```

```
        n = n / 10;
```

HUNDRED DAYS OF CODE

DAY 16

```
}
```

```
if (decreasing)
```

```
    printf("The digits are in decreasing order from left to right.\n");
```

```
else
```

```
    printf("The digits are not in decreasing order from left to right.\n");
```

```
}
```

```
/* Function to find largest and smallest digit */
```

```
void findLargestSmallest()
```

```
{
```

```
    int n, digit;
```

```
    int largest = 0, smallest = 9;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    while (n > 0)
```

```
    {
```

```
        digit = n % 10;
```

```
        if (digit > largest)
```

```
            largest = digit;
```

```
        if (digit < smallest)
```

```
            smallest = digit;
```

HUNDRED DAYS OF CODE

DAY 16

```
n = n / 10;
}

printf("Largest digit = %d\n", largest);
printf("Smallest digit = %d\n", smallest);
}

/* Function to find second largest and second smallest digit */
void findSecondLargestSmallest()
{
    int n, digit;

    int largest = -1, secondLargest = -1;
    int smallest = 10, secondSmallest = 10;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    while (n > 0)
    {
        digit = n % 10;

        /* Find largest and second largest distinct digits */
        if (digit > largest)
        {
            secondLargest = largest;
```

HUNDRED DAYS OF CODE

DAY 16

```
        largest = digit;
    }

    else if (digit > secondLargest && digit != largest)
    {
        secondLargest = digit;
    }

    /* Find smallest and second smallest distinct digits */
    if (digit < smallest)
    {
        secondSmallest = smallest;
        smallest = digit;
    }

    else if (digit < secondSmallest && digit != smallest)
    {
        secondSmallest = digit;
    }

    n = n / 10;
}

if (secondLargest == -1 || secondSmallest == 10)
{
    printf("At least two distinct digits are required.\n");
}

else
{

```

HUNDRED DAYS OF CODE

DAY 16

```
printf("Second largest digit = %d\n", secondLargest);

printf("Second smallest digit = %d\n", secondSmallest);

}

}

/* Main function */

int main()

{

    int choice;


    do

    {

        printf("\n===== MENU =====\n");

        printf("1. Check increasing order\n");

        printf("2. Check decreasing order\n");

        printf("3. Find largest and smallest digit\n");

        printf("4. Find second largest and second smallest digit\n");

        printf("5. Exit\n");

        printf("===== \n");


        printf("Enter your choice: ");

        scanf("%d", &choice);


        switch (choice)

        {

            case 1:
```

HUNDRED DAYS OF CODE

DAY 16

```
checkIncreasing();
```

```
break;
```

```
case 2:
```

```
checkDecreasing();
```

```
break;
```

```
case 3:
```

```
findLargestSmallest();
```

```
break;
```

```
case 4:
```

```
findSecondLargestSmallest();
```

```
break;
```

```
case 5:
```

```
printf("Program terminated.\n");
```

```
break;
```

```
default:
```

```
printf("Invalid choice! Please try again.\n");
```

```
}
```

```
} while (choice != 5);
```

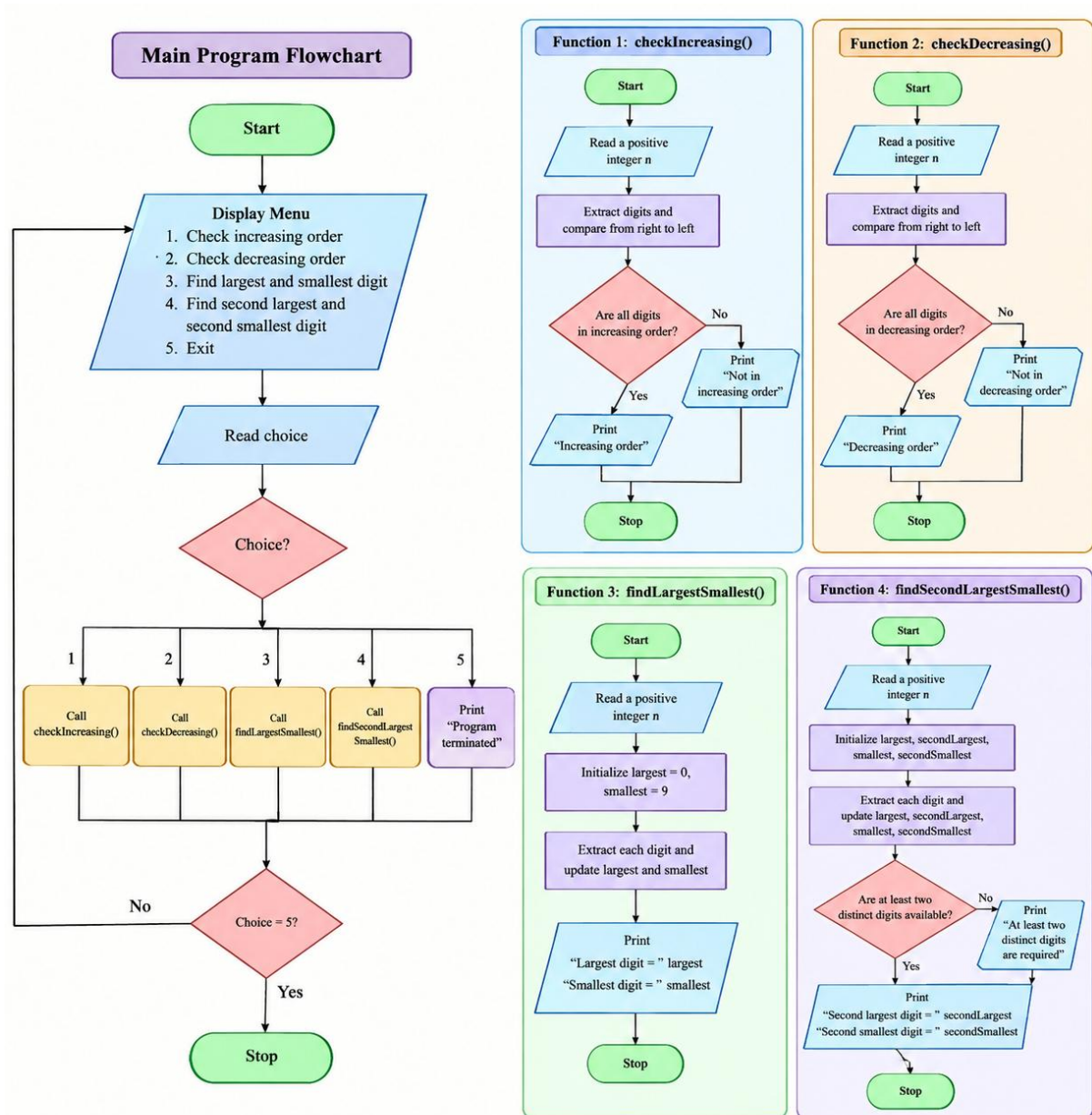
```
return 0;
```

```
}
```

HUNDRED DAYS OF CODE

DAY 16

FLOWCHART:



TEST CASE TABLE:

TEST CASE	MENU CHOICE	INPUT	EXPECTED OUTPUT
1	1	12345	Digits are in increasing order
2	2	98765	Digits are in decreasing order
3	3	90472	Second largest = 5, Second smallest = 2

HUNDRED DAYS OF CODE

DAY 16

4	4	58321	Second largest = 5, Second smallest = 2
5	5	---	PROGRAM TERMINATED

COMPILATION AND EXECUTION:

```
(kali㉿kali)-[~/day17]
└─$ vi order.c

(kali㉿kali)-[~/day17]
└─$ gcc order.c -o order

(kali㉿kali)-[~/day17]
└─$ ./order

===== MENU =====
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
=====
Enter your choice: 1
Enter a positive integer: 12345
The digits are in increasing order from left to right.

===== MENU =====
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
=====
Enter your choice: 2
Enter a positive integer: 98765
The digits are in decreasing order from left to right.
```

HUNDRED DAYS OF CODE

DAY 16

```
Enter your choice: 4
Enter a positive integer: 90472
Second largest digit = 7
Second smallest digit = 2
```

```
===== MENU =====
```

1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit

```
=====
Enter your choice: 3
Enter a positive integer: 58321
Largest digit = 8
Smallest digit = 1
```

```
===== MENU =====
```

1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit

```
=====
Enter your choice: 5
Program terminated.
```

```
(kali㉿kali)-[~/day17]
$
```